

Diagonal SSMs, Parallel Scan & HiPPO.

Three ideas turn the bare SSM recurrence into a fast, long-memory layer. **Diagonalising** A decouples the state into N independent scalar recurrences — cheap, and the basis of S4D/Mamba. The recurrence $h_t = a_t h_{t-1} + b_t$ is then an *associative scan*, so a parallel prefix computes all h_t in $O(\log T)$ depth instead of T sequential steps. And **HiPPO** initialises A so the state optimally memorises the input history by projecting it onto orthogonal polynomials — the trick that gives SSMs long-range memory. We prove the scan operator is associative, unroll a length-4 example sequentially and via the pairwise scan, and confirm they are bit-identical. Every number below is computed in full — nothing summarised or deferred.

THE EQUATION

$$h_t = a_t h_{t-1} + b_t \quad \Longleftrightarrow \quad (a_1, b_1) \circ (a_2, b_2) = (a_1 a_2, a_2 b_1 + b_2)$$

Worked example. $a = [0.5, 0.8, 0.3, 0.9]$, $b = [1, 0.5, 2, -1]$. We compute $h_{1:4}$ sequentially, then by combining the elements pairwise with $\circ$ in a balanced tree, and verify the two agree exactly. Plus the HiPPO-LegS matrix for $N = 4$, entries computed in full.

Topic 2 of 2 in this module · companion to the course page · v1.0

Contents

1	Diagonal state: decoupling the recurrence	2
2	The recurrence is an associative scan	3
3	Worked computation: sequential vs. tree scan	3
4	HiPPO: initialising A for long memory	4
5	Mamba: input-dependent (selective) SSM	5
6	Where this sits in the track	5
A	Reproducible (NumPy)	6

1 Diagonal state: decoupling the recurrence

Topic 1's discrete recurrence is $h_t = \bar{A} h_{t-1} + \bar{B} u_t$ with $\bar{A} \in \mathbb{R}^{N \times N}$. A dense $\bar{A}$ couples all N state coordinates: each $h_t^{(i)}$ depends on *every* $h_{t-1}^{(j)}$, an $O(N^2)$ matrix multiply per step. **Diagonalise** it: take $A = \text{diag}(\lambda_1, \dots, \lambda_N)$ (equivalently work in the eigenbasis of a diagonalisable A). Then $\bar{A} = \text{diag}(\bar{a}_1, \dots, \bar{a}_N)$ with $\bar{a}_i = e^{\Delta \lambda_i}$, and the recurrence *splits* into N *independent scalar* recurrences:

$$h_t^{(i)} = \bar{a}_i h_{t-1}^{(i)} + \bar{b}_i u_t, \quad i = 1, \dots, N \quad (1)$$

Cost drops from $O(N^2)$ to $O(N)$ per step, and the N channels parallelise trivially. This is the **S4D** simplification and the state form Mamba uses.

A diagonal A is N separate “leaky accumulators” running side by side, each with its own decay rate $\bar{a}_i$ and its own input gain $\bar{b}_i$. They never talk to each other inside the state; mixing happens only at the readout $y_t = C h_t = \sum_i C_i h_t^{(i)}$. Decoupling is what makes the layer cheap and makes the parallel scan below embarrassingly parallel across channels.

Each channel now has the form $h_t = a h_{t-1} + b_t$ (dropping the superscript, with $a = \bar{a}_i$ fixed and $b_t = \bar{b}_i u_t$). More generally we allow a *time-varying* a_t — Mamba does exactly this — giving the first-order linear recurrence we now scan.

2 The recurrence is an associative scan

A **scan** (prefix computation) turns a sequence of elements $e_1, \dots, e_T$ under a binary operator $\circ$ into all partial combinations $e_1, e_1 \circ e_2, e_1 \circ e_2 \circ e_3, \dots$. If $\circ$ is *associative*, those prefixes can be computed by a balanced binary tree in $O(\log T)$ *depth* and $O(T)$ total work — the Blelloch / Hillis–Steele parallel scan.

Encode each step as the pair $e_t = (a_t, b_t)$. Define the combine operator

$$(a_1, b_1) \circ (a_2, b_2) = (a_1 a_2, a_2 b_1 + b_2) \quad (2)$$

Why this is the right operator. Composing two recurrence steps: if $h' = a_1 h + b_1$ and then $h'' = a_2 h' + b_2$, substitute to get $h'' = a_2(a_1 h + b_1) + b_2 = (a_1 a_2)h + (a_2 b_1 + b_2)$. So the composition of the two affine maps $h \mapsto a_i h + b_i$ is again affine with parameters exactly (2). Scanning $e_1 \circ \dots \circ e_t$ therefore yields the affine map from h_0 to h_t ; with $h_0 = 0$ its b -component is h_t .

2.1 Associativity (proof)

Composition of functions is always associative, but we verify it on (2) directly. Let $e_i = (a_i, b_i)$. Left grouping:

$$(e_1 \circ e_2) \circ e_3 = (a_1 a_2, a_2 b_1 + b_2) \circ (a_3, b_3) = (a_1 a_2 a_3, a_3(a_2 b_1 + b_2) + b_3) = (a_1 a_2 a_3, a_2 a_3 b_1 + a_3 b_2 + b_3).$$

Right grouping:

$$e_1 \circ (e_2 \circ e_3) = (a_1, b_1) \circ (a_2 a_3, a_3 b_2 + b_3) = (a_1 a_2 a_3, a_2 a_3 b_1 + (a_3 b_2 + b_3)) = (a_1 a_2 a_3, a_2 a_3 b_1 + a_3 b_2 + b_3).$$

The two are identical, so $(e_1 \circ e_2) \circ e_3 = e_1 \circ (e_2 \circ e_3)$. ■

Associativity is the whole point. It means we may regroup the chain $e_1 \circ e_2 \circ \dots \circ e_T$ in *any* order — in particular as a balanced tree. A sequential scan is T steps of latency; the tree scan is $O(\log T)$ depth at $O(T)$ work. This is how an SSM layer trains in parallel over the time axis even when, as in Mamba, there is no convolution to fall back on.

3 Worked computation: sequential vs. tree scan

Elements $e_t = (a_t, b_t)$ with $a = [0.5, 0.8, 0.3, 0.9]$, $b = [1, 0.5, 2, -1]$, and $h_0 = 0$.

3.1 Sequential (ground truth)

$$\begin{aligned} h_1 &= 0.5(0) + 1 = 1.0, \\ h_2 &= 0.8(1.0) + 0.5 = 1.3, \\ h_3 &= 0.3(1.3) + 2 = 2.39, \\ h_4 &= 0.9(2.39) - 1 = 1.151. \end{aligned}$$

3.2 Tree scan via the combine operator $\circ$

Combine adjacent pairs first, in parallel:

$$\begin{aligned} p_{12} &= e_1 \circ e_2 = (0.5 \cdot 0.8, 0.8 \cdot 1 + 0.5) = (0.40, 1.3), \\ p_{34} &= e_3 \circ e_4 = (0.3 \cdot 0.9, 0.9 \cdot 2 + (-1)) = (0.27, 0.8). \end{aligned}$$

Then one more combine for the length-4 prefix:

$$p_{1234} = p_{12} \circ p_{34} = (0.40 \cdot 0.27, 0.27 \cdot 1.3 + 0.8) = (0.108, 1.151).$$

Read off the prefixes' b -components (each h_t is the b of the prefix $e_1 \circ \dots \circ e_t$):

$$\begin{aligned} h_1 &= b(e_1) = 1.0, \\ h_2 &= b(p_{12}) = 1.3, \\ h_3 &= b(p_{12} \circ e_3) = b((0.40, 1.3) \circ (0.3, 2)) = b(0.12, 0.3 \cdot 1.3 + 2) = 2.39, \\ h_4 &= b(p_{1234}) = 1.151. \end{aligned}$$

3.3 They are identical

t	a_t	b_t	h_t (sequential)	h_t (tree scan)
1	0.5	1.0	1.000	1.000
2	0.8	0.5	1.300	1.300
3	0.3	2.0	2.390	2.390
4	0.9	-1.0	1.151	1.151

Bit-for-bit identical — the scan is an exact reorganisation, never an approximation. The depth coloured below (intensity $\propto$ work at that tree level, capped **indigo!80**): a sequential scan would be 4 serial steps; the tree is 2 levels.

	level 1 (pairs)	level 2 (root)
combines	$e_1 \circ e_2, e_3 \circ e_4$	$p_{12} \circ p_{34}$

4 HiPPO: initialising A for long memory

A scan runs *any* A fast, but which A *remembers*? Random or naive A either forgets too fast or blows up. **HiPPO** (High-order Polynomial Projection Operators) chooses A so the state $h(t)$ holds the coefficients of the *best polynomial approximation of the entire input history* $u(s)$, $s \leq t$, against a measure. For the scale-invariant **LegS** (Legendre) measure, the matrix $A \in \mathbb{R}^{N \times N}$ has the closed form

$$A_{nk} = - \begin{cases} \sqrt{(2n+1)(2k+1)} & n > k \\ n+1 & n = k \\ 0 & n < k \end{cases} \quad n, k = 0, \dots, N-1. \quad (3)$$

It is lower-triangular (plus diagonal). Evaluating (3) for $N = 4$, the off-diagonal entries are $\sqrt{(2n+1)(2k+1)}$: e.g. $(n, k) = (1, 0) \rightarrow \sqrt{3} = 1.732051$, $(2, 1) \rightarrow \sqrt{15} = 3.872983$, $(3, 2) \rightarrow \sqrt{35} = 5.916080$. In full:

$$A_{\text{LegS}} = - \begin{bmatrix} 1 & 0 & 0 & 0 \\ \sqrt{3} & 2 & 0 & 0 \\ \sqrt{5} & \sqrt{15} & 3 & 0 \\ \sqrt{7} & \sqrt{21} & \sqrt{35} & 4 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ -1.732051 & -2 & 0 & 0 \\ -2.236068 & -3.872983 & -3 & 0 \\ -2.645751 & -4.582576 & -5.916080 & -4 \end{bmatrix}.$$

Think of each state coordinate $h^{(n)}$ as the coefficient of the n -th orthogonal (Legendre) polynomial in a running, online fit to “everything seen so far”. As new input arrives the coefficients update via $h' = Ah + Bu$ to keep the fit optimal — a continuously refreshed compressed summary of the past. A few coefficients capture the broad shape of a long history, which is precisely how a fixed-size state captures *long-range* dependencies instead of just the last few tokens. The structured A_{LegS} is the exact dynamics that make this projection self-consistent.

In practice S4/S4D *diagonalise* the HiPPO matrix (its normal-plus-low-rank structure has a known near-diagonal form), inheriting the long-memory initialisation while keeping the cheap diagonal scan of Section 1.

5 Mamba: input-dependent (selective) SSM

Classical S4 keeps A, B, C, Δ *fixed* across time, so $a_t = \bar{a}$ is constant and the model is exactly the convolution of Topic 1. **Mamba** makes B, C, Δ (and hence $\bar{A} = e^{\Delta A}$, $\bar{B}$) *functions of the input* u_t — “*selective*” state spaces. Now a_t varies with t , so the model can choose, per token, how much to remember versus overwrite (e.g. reset the state at a delimiter, or hold a value across a long gap). This data-dependence is what lets a fixed-size state behave adaptively and closes much of the recall gap against attention.

Selectivity has a price. With time-varying a_t, b_t the kernel $\bar{K}_k = C\bar{A}^k\bar{B}$ is no longer a single fixed sequence — there is *no convolution form* (Topic 1’s view (ii) dies). Mamba therefore *cannot* train via FFT convolution; it relies entirely on the parallel *scan* of Section 2, which still works because the per-step affine maps (a_t, b_t) compose associatively even when they differ each step. Mamba’s engineering effort goes into a hardware-aware scan (kept in fast SRAM, fused) precisely because the convolution shortcut is gone.

6 Where this sits in the track

This closes Module 7.2. Putting the pieces together:

ingredient	what it buys	cost
diagonal A (S4D)	N decoupled scalar recurrences	$O(N)/\text{step}$
associative scan	parallel training, no convolution needed	$O(\log T)$ depth, $O(T)$ work
HiPPO init	optimal long-range memory of history	structured A
Mamba selectivity	input-dependent, adaptive memory	scan-only (no conv.)

Together they give a sequence layer that is $O(T)$ at training (via scan), $O(1)$ -state at inference (the recurrence), and competitive with attention on long-range tasks — the headline promise of modern SSMs.

A Reproducible (NumPy)

Inputs: $a = [0.5, 0.8, 0.3, 0.9]$, $b = [1, 0.5, 2, -1]$ for the scan; HiPPO-LegS via (3) for $N = 4$. The snippet runs the sequential scan, the pairwise tree scan, checks they match, and builds A_{LegS} .

```
import numpy as np
a=[0.5,0.8,0.3,0.9]; b=[1.,0.5,2.,-1.]
comb=lambda L,R:(L[0]*R[0], R[0]*L[1]+R[1])    # (a1,b1)o(a2,b2)
h=0.; seq=[]
for at,bt in zip(a,b): h=at*h+bt; seq.append(h)    # [1.0,1.3,2.39,1.151]
e=list(zip(a,b))
p12=comb(e[0],e[1]); p34=comb(e[2],e[3])          # (0.4,1.3) (0.27,0.8)
tree=[e[0][1], p12[1], comb(p12,e[2])[1], comb(p12,p34)[1]]
print(np.round(seq,3)); print(np.round(tree,3))    # identical
N=4; A=np.zeros((N,N))
for n in range(N):
    for k in range(N):
        A[n,k]=-(np.sqrt((2*n+1)*(2*k+1))) if n>k else (n+1) if n==k else 0.
print(np.round(A,6))    # HiPPO-LegS; A[1,0]=-1.732051, A[2,1]=-3.872983
```